Introduction

SOFTWARE ENGINEERING -

- Systematic approach of designing, developing, testing and mainlaining software by applying engineering principles.

Need -

(i) Software is growing in size and complexity.
(ii) Simple coding is not enough for large projects.
(iii) Ensures reliability, low cost, and timely delivery.

Importance

(i) Reduces software failures.
(ii) Improves productivity and customer satisfaction.
(iii) Save time and cost through planning and reuse.

Application - Banking, e-commerce, education, Healthcare etc.

Importance of Software Engineering as a Discipline

(i) Provides structured methods to handle large and complex software projects.
(ii) Ensures software is reliable, efficient, and user-friendly.
(iii) Reduces cost & Time through planning, estimation and reuse of components.
(iv) Delivers software that meets user requirements and expectations.
(v) Support teamwork among developers, testers, designers and clients etc.
(vi) Uses systematic testing and verification to reduce failures.

Software Applications

(i) System App^n :-
- Helps the computer run efficiently.
- Eg – Operating Systems (Windows, Linux), Utility Software (antivirus, file managers).

(ii) Business App^n :-
- Used by organizations for managing work.
- Eg – ERP systems, Banking software, e-commerce platforms

(iii) Web App^n :-
- Accessed via browsers over the Internet.
- Eg – Gmail, Google Docs, Amazon, social media apps.

(iv) Mobile App^n :-
- Apps designed for smartphones and tablets.
- Eg – WhatsApp, Instagram, Paytm, Uber.

(v) Embedded App^n :-
- Built into hardware devices to control them.
- Eg – Software in washing machines, cars, ATMs.

(vi) Entertainment App^n :-
- Provide fun and entertainment.
- Eg – games, music players, video streaming apps.

Software Crisis

- Problem faced in early days of software development.
- Software becomes large and complex, but methods were not advanced.
- The software crisis is the difficulty of building large, reliable, and efficient software on time and within budget, which led to the birth of Software Engineering.

Main Problem in Software Crisis

(i) Over Budget – Software projects often cost more than planned.

(ii) Time overruns – Projects were not completed on schedule.

(iii) Low quality – Software was unreliable, had many bugs, or failed.

(iv) Poor Maintainability – Difficult to update and modify software later.

(v) Mismatch with Requirements – Final software often did not meet user needs.

(vi) Productivity issues – Developers struggled to manage large, complex ~~objects~~ projects.

Reason: Lack of proper methods, tools and planning.

Solution: Rise of Software engineering as a discipline.

Software Process

- A software process is the set of activities and steps followed to develop software.
- It is like a roadmap for building software from idea → delivery → maintenance.

Main activities in Software Process :-

(i) Requirement Analysis → Understanding what the user wants.

(ii) Design – Planning how the software will work.

(iii) Implementation – Writing the actual program (coding).

(iv) Testing – Checking and fixing errors / bugs.

(v) Deployment – Delivering software to users.

(vi) Maintenance – Updating and improving software later.

Characteristic of Software Process –

(i) Systematic – follows a structured and step-by-step approach.

(ii) Repeatable – can be applied to different projects in the same way.

(iii) Measurable – Progress, cost and quality can be tracked.

(iv) Predictable → outcomes (time, effort, results) can be estimated.

(v) Maintainable – supports updates and improvements later.

(vi) Quality-oriented – Ensures reliable and error-free software.

(vii) Flexible – can adapts to changes in requirements or technology.

(viii) Goals-directed – Aims to deliver software within time, cost, and quality limits.

SDLC

SOFTWARE LIFE CYCLE – DEVELOPMENT

- Structured process that is used to design, develop, and test high-quality software.
- SDLC, is a methodology that defines the entire procedure of software development step-by-step.
- The goal of the SDLC life cycle model is to deliver high-quality, maintainable software that meets the user's requirements.

Stages of SDLC – 6 stages.

(i) Requirement Analysis
(ii) System Design
(iii) Implementation
(iv) Testing
(v) Deployment
(vi) Maintenance

1) Requirement Analysis –

- Understand what the users or clients need from the software.
- Identify functional requirements (what system should do) and non-functional requirements (performance, security, usability).
- Prepare a Software Requirement Specification (SRS).

* SRS – A SRS is a document that clearly describes what the software system should do and how it should perform.
  - It acts as a contract b/w the client and developers.

② Designing –

- Plans and design overall structure of the ~~program~~ software.
  - ↳ High-level design – defines system architecture and modules.
  - ↳ Detailed design – specifies how individual components will work.
- Provides a clear roadmap for coding and future testing.

③ Implementation (coding / Development) –

- Developers write the code according to the design documents.
- Use programming language, frameworks, and tools.
- Each module is built separately and then integrated.

④ Testing

- Check whether the software works correctly and efficiently.

Types of Testing:

- Unit Testing – test individual modules.
- Integration Testing – test combined modules.
- System Testing – test the complete system.
- User acceptance Testing (UAT) – Check if it meets the user expectations.

(v) Deployment –

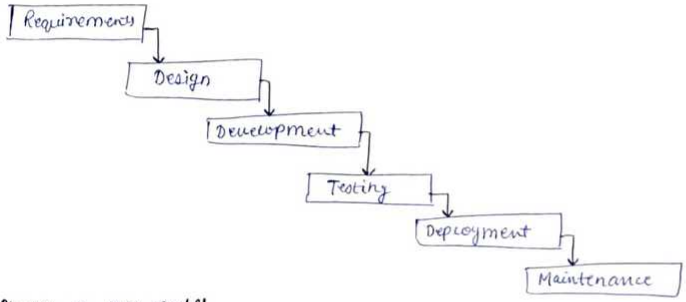
– Process of making the software available for use by the end-users in a real environment.

(vi) Maintenance

– Keep the software up-to data and bug-free.

– Handles changes in requirements or technology.

WATERFALL



Phases of this model

(i) Requirement – gathering requirement from stakeholders and analyzing them to understand the scope and objectives of the project.

(ii) Design – Create detailed design document that outlines the software architecture, user interface, and system components.

(iii) Development – Include implementation involves coding the software based on design specification.

(iv) Testing – Software is tested as a whole to ensure that it meets the requirements.

(v) Deployment – Software is tested and approved, it is deployed to the production environment.

(vi) Maintenance – Fixing any issues that arises after the software has been deployed and ensuring that it continues to meet the requirements over time.

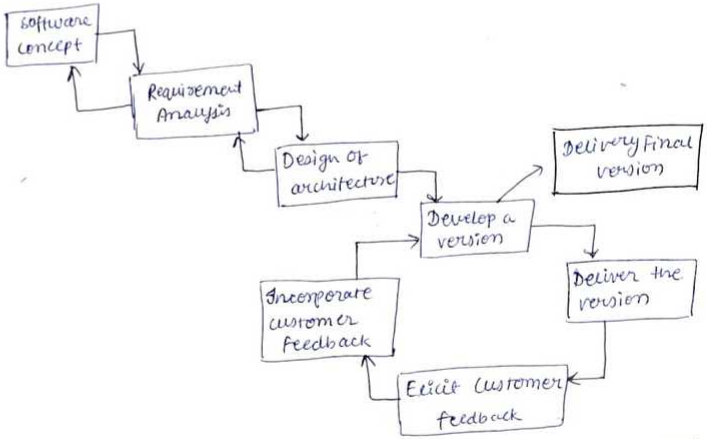
Limitation – No backtracking is allowed.

Advantage – Simple and easy to use, especially for small projects.

Disadvantage – Not flexible, hard to make changes once a phase is completed.

EVOLUTIONARY MODEL

- divides development into smaller incremental cycles (~~mini~~ mini-waterfall)
- Users get access to the product at the end of each cycle.
- Feedback from the users is used to plan the next cycle.
- Software evolves over time with continuous improvements.
- Work is broken into smaller, manageable chunks for delivery.
- Supports changing requirements during development.



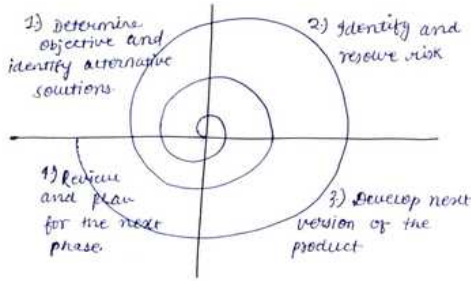
Advantages → Increases customer confidence as they see working software early and often.

Disadvantage → Can be time-consuming and costly if there are too many iterations.

Limitation- Requires continuous user involvement, which may always be feasible not

SPIRAL MODEL

- Combination of waterfall model and iterative model.
- A systematic and iterative software development model where development progress in spiral loops rather than sequential steps
- Diagram- Looks like a spiral with multiple loops, each representing a phase of development
- Risk- driven model.



Phases

(i) Objectives defined (Planning Phase)
→ Collect requirement from customer.
→ Define objectives – What the project must achieve (both functional and non-functional).
→ Identify different possible solutions for the given requirements.

(ii) Risk Analysis & Resolution
→ Analyze all possible solution chosen in phase 1.
→ Identify risks (technical, financial, time, user acceptance).
→ Build a prototype to test and reduce risks.
→ Select the best feasible solution after evaluation.

(iii) Development and Testing (Engineering Phase)
→ Develop the next version of the product
→ Implement features decided in earlier phases.
→ Perform testing and verification to ensure quality and requirement satisfaction.
→ Deliver a working version to the customer.

(iv) Review & Planning for the Next Iteration (Evaluation Phase)
→ Customer reviews the developed version.
→ Take feedback from the customer.
→ Plan next iteration based on results and risk.
→ Start a new loop of the spiral until the final product is complete.

Advantage- Handles risk very well and allows changes at any stage.

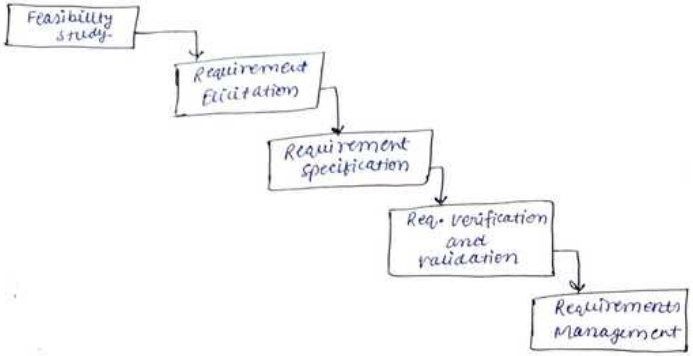
Disadvantage- Expensive and time-consuming.

Limitation- Needs skilled risk managers, not suitable for small or low-budget projects.

Software Requirements Analysis & Specification

1) Requirement Engineering –
- Process of collecting, analyzing, documenting, and managing requirements of a software system.
- Ensures the software meets user needs.
- Steps usually include –
  - Elicitation → Gathering requirements from stakeholders.
  - Analysis → Checking requirements for conflicts, feasibility
  - Specification → Writing requirements in clear document (SRS)
  - Validation → Making sure req. are correct and complete.

Requirement Engineering Process



Functional Requirement

(i) Define what the system should do

(ii) Behaviour and functions of the system.

(iii) Eg - data input, output, processing, authentication etc.

(iv) We can measure it easily

(v) Related to actions and operations of the system

(vi) Directly visible to users.

(vii) Verified by checking outputs of functions.

Non-functional Requirement

(i) Define how the system should work.

(ii) Based on performance, security, quality of the software etc.

(iii) Eg - reliability, scalability, security etc.

(iv) It is difficult to measure.

(v) We define constraints and conditions under the system must operate

(vi) Not directly visible, but affect user experience

(vii) Verified by measuring performance, usability etc.

System Requirements -

- Detailed description ~~of the~~ of what a system (software, hardware) needs in order to work properly.

- They cover both functional and Non-functional needs, but written in a more technical and specific way than user requirements

Types of System Req.

(i) Hardware Requirements

→ Physical devices needed to run the system.

Eg → Processor speed, RAM size, storage, network devices

(ii) Software Requirements

→ Software platforms, OS, libraries, or databases required.

Eg → Windows / Linux OS, MySQL database, Python / Java runtime.

(iii) Functional System Requirements

→ What the system must do at a technical level

Eg - System must generate monthly sales reports.

(iv) Non-Function System Requirements

→ How the system should perform.

Eg - System should support 1000 users at the same time.

Requirement Elicitation Techniques

→ These are different methods or approaches used to gather requirements from stakeholders, users, and customers in order to understand what the software system should do.

① FAST (Facilitated Application Specification Technique) (12)

- A meeting best approach where stakeholders (users, customers, developers, manager) come together.

Uses brainstorming and group discussions to gather requirements.

Helps in avoiding misunderstanding because everyone is directly involved.

BEST FOR: Projects where clear communication among all parties is needed.

② QFD (Quality function Deployment)

→ A technique to translate customer needs into technical req. using structured diagrams.

(i) Works
Collect customer needs ("what the customer wants").

(ii) Map them into technical req. ("How to achieve it")

③ Use case Approach

→ A way to capture req. by describing how users (actors) interact with the system

Actor– people or external system interacting with the s/w.

Scenario– step-by step description of how a task is done.

Use-case diagram– visual representation showing actors and system interactions.

Characteristics of SRS (13)

(1) Describe the system exactly as needed.
(2) Includes all req. (fun. + non fun.)
(3) Unambiguous– only one meaning, clear, no confusion
(4) non conflicting req.
(5) req. can be tested later
(6) easy to change/update when needed.
(7) each req. can be tracked (from source to implementation)

Organization SRS

Introduction– purpose, scope, definitions.

overall description– features, user, constraints, assumptions

Specific req. – Func. + Non func.

External Interfaces– UI, h/d, s/w.

R.g using DFD

→ During rq, a DFD is used to model how data moves through a system

Helps in understanding i/p, processes, o/p, and data storage clearly

Process (O/O)– work or action.

Data flow (→) – movement of data

Data store (=) :- Storage

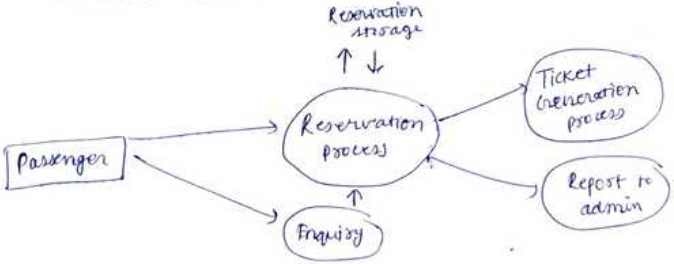
External entity (□) – outside system

① Level 0 (Context diagram)

Highest level → shows the system as a single process with I/O to external entities.



② Level 1 — Breaks main process into sub-processes.



③ Level 2

→ Further details of level 1 processes.

• Data dictionary

A structured list of all data items in the system. Contains details like Name of data items, Type (int, string, etc), Source & destination, description/meaning

SOFTWARE ENGINEERING

UNIT-2

1) Software Project Planning :

(a) Size Estimation like lines of code & function count.

(b) Cost Estimation Model

(c) COCOMO

(d) Putnam resource allocation model (app^n & limitation)

(e) Validating Software Estimate

(f) Risk Management

2) Software Design :

(a) Cohesion & coupling (impact)

(b) classification of cohesiveness & coupling

(c) function oriented design. (data & disk)

(d) object oriented Design

(e) User interface Design.

PYQ

Q) Live variables and module Weekness.

difference BW FC & LOC.

Intermediate COCOMO

① Software Project Planning

a) Software Project Planning = a roadmap for software development to deliver the project on time, within budget, and with good quality.

It answers questions like :-

- What to do? (Scope)
- How to do it? (Methods & tasks)
- When to do it? (Schedule)
- How much will it cost? (Budget)

We need to estimate its size, so we can plan time, cost, and efforts.

Two common methods are:

- Lines of code
- Function point (FP) count

① LINES OF CODE :-

- LOC count the total no. of lines of source code in a project.
  The units of LOC are :-
  KLOC - Thousands lines of code.
  NLOC - Non comments lines of code.
  KDSI - Thousands of Delivered source instruction.
- Eg - A calculator app may need 2000 LOC, while an OS may needs millions of LOC.

Features :-

→ LOC can be tracked over time to see project growth or reduction

→ Same LOC programs can have very different complexity.

Advantages -

- Effort estimation :- can give rough time & effort needed.
- Comparison :- Helps compare productivity b/w projects / team.

Disadvantages -

- Bigger LOC = harder to maintain.
- Doesn't fit ~~for~~ well with changing requirements in agile.

Function Count

- Function count is a technique to estimate the size (complexity, functionality) of software system based on the functions it provides to the user, not just line of code
- It measures software in term of functionality delivered, independent of technology or programming language.

Function Points	Lines of code
(i) Measures the functionality delivered to the user based on user req. and design.	Measures the physical lines of source code written in a programming language.
(ii) User functionality and system behavior	The amount of code written
(iii) Independent of programming language, platform and technology used.	Dependent of prog. lang.
(iv) Based on user inputs, outputs, inquiries files and interfaces.	Based on counting the lines of code written

Unadjusted function point (UFP) – Calculate UFP by multiplying the no. of each type of component by its corresponding weight and summing them up. Factors = 5

Value Adjustment Factor (VAF) – A factor that adjusts the UFP based on general system characteristics (GSCs), F = 14.

FP = UFP × (0.65 + 0.01 * Sum of GSCs)

= UFP × VAF

5 for UFP

(1) External Input – (EI) (3)
– User registration form, product search form, add to cart form

(2) External output (EO) (1)
– confirmation email, order summary page, invoice generation

(3) External Inquiries (EI) (2)
– Product Availability check, order status check

(4) Internal logical files (ILF) (3)
– User data, product catalog, order data.

(5) External interface files (EIF) (2)
– Payment gateway interface, 3rd party shipping

component	<5 Low	5-15 Avg	>15 high complexity.
EI	3	4	6
EO	4	5	7
EQ	3	4	6
ILF	7	10	15
ELF	5	7	10

5 UFP factor value are less than 8 each (expect some) that means less complexity.

5 × 3 = 9 , 3 × 4 = 12 , 2 × 3 = 6 , 3 × 7 = 21 , 2 × 5 = 10

9 + 12 + 6 + 21 + 10 = 58

FP = UFP (58) + (0.65 + 0.01 × Sum of GSCs)

Q.

		Weighting factor
EI	30 × 4 = 120	4
EO	60 × 5 = 300	5
EQ	23 × 4 = 92	4
ILF	08 × 10 = 80	10
ELF	02 × 7 = 14	7
UFP =	606	

Out of 14 — 4 (factors not applicable)

14 − 4 = 10

4 → value = 3 ; 6 → value = 4

(4 × 3) + (6 × 4) = 36

FP = UFP × VFP

= 606 × (0.65 + 0.01 × 36)

= 606 × 1.01

= 612.06 Ans

	Low	Avg	High
EI	3	4	6
EO	4	5	7
EQ	3	4	6
ILF	7	10	15
ELF	5	7	10

Q.

EI = 30 × 4 = 120

EO = 42 × 5 = 210

EQ = 08 × 4 = 32

ILF = 07 × 10 = 70

ELF = 30 × 7 = 210

UFP = 642

FP = UFP × VAF

= 642 × (.65 + .01 × 14 × 2)

= 642 × 0.93

= 597.06 FP.

Cost estimation model = methods to calculate software cost, effort, and time

Major model = Empirical, Heuristic, Analytical, Hybrid

Eg = COCOMO model, Putnam's model, FP.

FAST = Facilitated Application Specification Technique

QFD – Quality function deployment
Technique to translate customer needs into technical requirements

Amplified prototype – Use case approach
Already 80% captured req. by analyzing key users (actors) interact with the system

COCOMO MODEL

(i) Project size
(ii) Effort required to complete the project
(iii) Project duration
(iv) Cost

3 modes of COCOMO model
1. Organic mode
2. Semi-detached mode
3. Embedded mode

1. Basic COCOMO
2. Intermediate COCOMO
3. Complete COCOMO

BASIC COCOMO

$Effort = a_1 \times (KLOC)^{a_2}$ PM → person month

$T_{dev} = b_1 \times (effort)^{b_2}$ months.

Software projects	a_1	a_2	b_1	b_2
organic	2.4	1.05	2.5	0.38
semidetached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Estimation of development effort

Organic : $Effort = 2.4(KLOC)^{1.05}$ PM

Semidetached : $Effort = 3.0(KLOC)^{1.12}$ PM

Embedded : $Effort = 3.6(KLOC)^{1.20}$ PM

Development time

$T_{dev} = 2.5(effort)^{0.38}$ months

$2.5(effort)^{0.35}$ m

$2.5(effort)^{0.32}$ m

Q. Semi detached mode 30,000 LOC

30 KLOC

$Effort = a_1 \times (KLOC)^{a_2}$ PM

$= 3.0(30)^{1.12}$

$= 3.0(45.2)$

$Effort = 135$ PM

$T_{dev} = 2.5(effort)^{0.35}$

$= 2.5(135)^{0.35}$

$= 2.5(5.56)$

$T_{dev} = 13.9 = 14$ months

Suppose 1 person की salary = 10000 ₹/

amount → 10000 × 14 = 140,000

Avg / Person estimation & staff size $N = \frac{E}{D} = \frac{135}{14} = 9.6 \approx 10$ persons approx

Productivity (P) = $\frac{KLOC}{E}$ KLOC/PM = $\frac{30000}{135} = 222.23$

Let's say, ques में आपको determine करना है की organic, S-D, Embedded project size दिया होगा / जैसे PS = 250 KLOC है तो semi detached.

नीचे chart से देख लो

Organic	2-50 KLOC
Semi detached	50-300 KLOC
Embedded	300 KLOC

Intermediate COCOMO

→ Uses a set of 15 cost drivers.

$E = a_1(KLOC)^{b_1} \times (EAF)$ → Effort Adjustment factors (given)

$T_{dev} = b_1(E)^{b_2}$

Software projects	a_1	b_1
organic	3.2	1.05
semi detached	3.0	1.12
~~COCOMO~~ Embedded	2.8	1.20

Eg- 30,000 LOC, which is embedded software with critical area hence reliability high. (1.15)

$E = a_1(KLOC)^{b_1} \times EAF$

$= 2.8(30)^{1.20} \times 1.15$

$= 191$ PM

$T_{dev} = b_1(E)^{b_2}$

$= 2.5(191)^{0.32}$

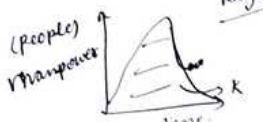
$= 13$ months

$N = \frac{E}{D} = \frac{191}{13} = 15$ person approx

Putnam model

→ No. of people required over time to time.

Rayleigh curve



$\sqrt{e} = 1.6$

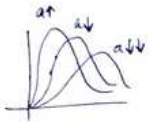
cohesion	coupling
measures how strongly elements of a module belong together	measures degree of dependency b/w two diff modules.
High cohesion ✓ within modules	Low coupling ✓ B/w modules

The eqn of above curve represents manpower, measured in persons per unit time as a function of time

$$m(t) = \frac{dy}{dt} = 2kat\,e^{-at^2} \quad *$$

dy/dt = manpower utilization rate per unit time.

t = elapsed time (शिखरता time बीत चुका $\frac{2}{\sqrt{e}}$)



a = parameter ($1/time^2$)
(graph की shape की change करने में)

K = area under curve $[0, \infty]$.

$$y(t) = K\left[1 - e^{-at^2}\right] \quad *$$

y(t) → cummulative manpower

K = constant

$y(\infty) = K$

$y(0) = 0$

$$t_d^2 = \frac{1}{2a}$$

t_d = time where maximum rate occurs. (peak development time) also total project development time.

$$E = K(1 - e^{-0.5})$$

$$E = 0.3935K \quad *$$

Estimate of dev. time

K = total project cost

$$M_o = \frac{K}{t_d \sqrt{e}} \quad *$$

M_o = peak manning

$\sqrt{e} = 1.648$

t_d = delivery time

Avg rate of software team build up: $\frac{m_o}{t_d}$ *

- All time is calculated in years.

Q. A software project is planned to cost 95 PY in a period of 1 year 9 months.
Calculate peak manning and avg rate of software team build up.

95 PY ⇝ person year = K

$K = 95$

$\sqrt{e} = 1.6$

$t_d = 1 + \frac{9}{12} = 1 + \frac{3}{4} = 1.75$

$$m_o = \frac{K}{t_d \sqrt{e}} = \frac{95}{1.75 \times 1.6} = \frac{95}{2.8} = 33.9 \approx 33 \text{ person}$$

$$\text{Avg rate} = \frac{m_o}{t_d} = \frac{33.9}{1.75} = 19.3$$

Risk management

→ process of identifying, analyzing, and handling risks that may negatively affect a software project's success (in terms of cost, schedule, quality or performance).

STEPS

(i) Risk Identification

- Find possible risks before they occur.

Eg- technology risk, requirement changes

(ii) Risk Analysis (Assessment)

- Analyze each risk's possibility (likelihood) and impact
- Helps in prioritizing risks

(iii) Risk Planning (Mitigation Strategy)

- Decide actions to minimize or avoid risks

Eg- Backup staff training, prototyping for unclear req.

(iv) Risk Monitoring and control

→ continuously track risks during the project

→ New risk may arise → update plan

Types of Risk

- Project Risks – affect schedule & resources (deadline missed, cost overrun)
- Technical Risks – related to technology (software/hardware failure, integration issues)
- Business Risks – affect overall success (product not meeting customer needs)

COHESION	COUPLING
(i) Measures how strongly elements of a module belong together.	(i) Measures the degree of dependency b/w two different modules
(ii) Within a module	(ii) Between modules
(iii) desired level should be high	desired level should be low.
(iv) High cohesion – modules are focused, reusable, easy to maintain.	(iii) Low coupling – modules are independent, flexible, and easy to modify.
(v) Low cohesion – modules become complex, hard to understand.	(iv) High coupling – modules become independent, harder to maintain

Function-oriented design (FOD)

- A design method where the system is divided into functions (procedures)
- Focus is on what system does. (operations)

Adv

① Simple and easy to understand
② Good for small, less complex system
③ Func can be ~~not~~ reused in diff. programs.

disd

① Data is not well protected
② Difficult to maintain and modify for large systems

Object-oriented design (OOD)

→ A design method where system is divided into objects (data + classes + methods)

Adv

① Better data security (encapsulation)
② Easier to maintain, extend, and modify.
③ Promotes reuse through inheritance and polymorphism

DIS

① More complex to design and understand initially.
② can be slower execution compared to FOD.

UI

UI is the process of designing the part of software that users interacts with directly (screens, buttons, menus, forms) to make system easy, efficient, pleasant to use.

The software becomes most popular if its UI is —

→ Attractive
→ simple to use
→ responsive in short time
→ Clean to Understand.

UI is divided in 2 categories -

① Command line Interface
② Graphical User Interface.

Function oriented Design	Object Oriented Design
① Focuses on functions ~~and~~ or Procedures	① Focuses on Objects.
② Data and functions are separate.	② Data and methods are combined in objects.
③ Top-down approach	③ Bottom up Approach
④ Uses flowchart and DFD.	④ Uses class diagram and UML.
⑤ Changes in data may affect many function	⑤ Changes are easier due to encapsulation.
⑥ Less secure data handling. Eg- C, Pascal.	⑥ Better data security. Eg- Java, C++, Python etc.

UNIT-3

1. Software Metrices :
2. Software Measurement :
3. What & Why,
4. Token Count
5. Halstead Software Science Measures
6. ~~Data~~ Data structure Metrices.
7. Information Flow Metrices.
8. Software Reliability :
9. Importance
10. Hardware Reliability & Software Reliability
11. Failure and Faults
12. Reliability Models -
13. Basic Model
14. Logarithmic Poission Model
15. Software quality Models
16. CMM & ISO 9001.

SOFTWARE METRICES –

→ Software Metrices are numbers or measurements that helps us understand the quality, size, cost, performance, and progress of a software project.

Eg – How many lines of code?
How many bugs?
How fast is the program?
How complex is the code?

Matrix helps in: Planning, Estimation, Quality control, Improvement.

Software Measurements (What & Why) –

What is Software Measurement?

→ It is the process of assigning numbers/values to different properties of software.

Example –

- Size of code.
- Time taken to develop
- Errors found
- Functions implemented

Why do we measure software?

(i) To estimate cost, time, and effort
(ii) To track progress.
(iii) To improve quality.
(iv) To identify problems early.
(v) To ensure software meets requirements.

Token count –

→ Token count is a software size measurement that counts the total number of tokens in a program.

- Token –

A Token is a basic element of a programming language, such as:

- keywords (if, while, for)
- identifiers (a, sum, total)
- operators (+, −, =, *, /)
- constants (10, 3.14)
- Punctuation (;, (), {})

Eg – sum = a + b;

- sum
- =
- a
- +
- b
- ;

Token count = 6.

Halstead Software Complexity

- In 1977, Maurice Howard Halstead introduced metrices to measure software complexity.
- It is a method to measure how big, difficult, and complex a program is by counting the operators and operands in the code.
- These measurements are based on counting tokens in the program

Halstead counts two things -

(i) Operators - These are symbols or keywords that do something, like:

+, −, *, =, if, while, return

Eg → = and + are operators

(ii) operands = These are values or names, like:

x, y, sum, 10, a, b

Eg → a and b and sum are operands.

He defines various indicators to check complexity of a program -

n_1 = Number of unique operators

n_2 = No. of unique operands

N_1 = No. of total occurence of operator.

N_2 = No. of total occurence of operands.

Metrics represents as :-

Vocabulary, $n = n_1 + n_2$

Size, $N = N_1 + N_2$

Volume, $V = \text{size} * \log_2 \text{Vacabulary}$ → $V = N \times \frac{\log n}{\log 2}$

$N + \log_2 n$

Difficulty, $D = \frac{n_1}{2} * \frac{N_1}{n_2}$

E = Efforts = Difficulty * volume

Error B = $\frac{\text{Efforts}}{3000}$

T = Testing Time = $\frac{\text{Efforts}}{S}$ } S = 18 sec.

Consider the program,

```
Void Sort (int * a, int n){
    int i, j, t;
    if (n<2) return;
    for (i=0; i<n-1; i++){
      for (j=i+1; j<n; j++){
        if (a[i] > a[j]){
          t = a[i];
          a[i] = a[j];
          a[j] = t;
        }
      }
    }
}
```

n_2 = (No. of operands) - i, j, t, n, a, 1, 2, 0 = 8

n_1 = (No. of operators) - (,), {, }, [,], <, −, ++, +, >, =, if, int, for, return, void = 20

N_2 = No. of total occurence of operands, i=3, j=7, t=2, n=3, a=6, 1=2, 2=1, 0=1

→ 30.

N_1 = No. of total occurence of operators

int = 3, if = 2, return = 1, for = 2, void = 1.

=3, (=5,)=5, * =1, ,=3, {=4, }=4, ;= 9, <=3, >=1, [=6,]=6, −=1, ++=2, +=1.

Total N_1 = 63.

Data Structure Metrics

↳ measurement

S/w develop → process data

→ Data structure metrics are measurements used to check how good, efficient, or suitable a data structure is for solving a problem.

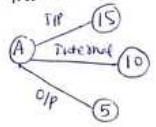
- Just like software metrics measures program quality,
- data structure metrics measure the performance of a data structure

Software Planner

Input	Internal / Intermediate data	Output
• cost	• cost of each developer	• Total duration
• duration	• other factors (circumstances, cond^n of our organization)	• Total effort
• No. of developer		• cost

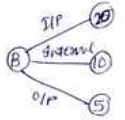
→ A count of the amount of data input to, processed in, and output from software is called Data structure metrics.

Choose the method:-



✓

- Small input
- less chance of errors



- More chances of errors

n_2 = VARS + unique constants + labels

(variables) a, b, c, d ; constants 0·1, 0·2, 0·3 ; labels goto

$N_2 = n_1 v_1 + n_2 v_2 + n_3 v_3 + n_4 c_1 + n_5 c_2 + n_6 l_1 + n_7 l_2$

↓ (No. of Times)

```
0  # include <stdio.h>
1  void main() {
2      int hours, rate, gross;
3      float pf, net;
4      scanf ("%d %d", &hours, &rate);
5      gross = hours * rate
6      pf = 0.1 * gross;
7      net = gross - pf;
8      printf ("%d %d", gross, net);
9  }
```

VARS = hours, rate, gross, pf, net

constant = 1

$n_2 = 5 + 1 + 0 = 6$

$N_2 = \underbrace{3+3+5+3+3}_{\text{VARS}} + \underbrace{1}_{c} + \underbrace{0}_{l} = 18$

live Variables

→ A variable is live at same point from its first reference to last reference.

```
int a, b;
a = 2; b = 1;   → b is live only in this line.
|||||           | a is live
|||||           | till this
a = a+2;        | part
```

A variable whose current value will be used in the future during program execution.

Eg →
```
x = 4;
y = x * 2;      // x is live
z = y + 1;      // y is live
return z;       // z is live until returned.
```

Live variable = $\frac{\text{Sum of count of live variable}}{\text{No. of executable statements}}$

Variable span :- The portion of a program during which a variable remains live (i.e its value is needed).

↓

The distance from where a variable is defined to where it is last used.

Eg :-
```
x = 4;          // definition of x
y = x * 3;      // last use of x
z = y + 2;      // last use of y
return z;       // last use of z
```

- n statements = n-1 span

Eg -
```
scanf ("%f %f", &x, &y);
m = x;
n = x - y;
printf("%f %f", x, y);
```

x = 4 statement
4 - 1 = 3 spans

y = 3 statements
3 - 1 = 2 spans

Information Flow Metrics :-

Size - LOC, FC, TC
Data structure metrics - n_1, n_2
] used for cost / time / Effort. Size Estimation method



Components – An element, identified by decomposing a software

Cohesion – means how closely the things inside a module are related to each other.

Eg – If a module does only one job, it has high cohesion (GOOD).
If it does many unrelated jobs, it has low cohesion (BAD).

Coupling – means how much one module depends on another module.

Eg – If modules are independent, coupling is low (GOOD).
If modules need each other a lot, coupling is high (BAD)

Definition –

→ Information flow Metrics are used to measure how data moves through a program – how functions receive, process, and send information.

→ They help estimate complexity, testing effort, and maintenance effort.

Why should we use Information flow Metrics?

- To understand how much a module depends on other module.
- To measure complexity of a module.
- To predict maintenance cost.

- To identify modules that are highly dependent → risky & hard to change.

Important Terms –

(i) Fan-in (FI) – No. of components that can call A.
– No. of modules that can call a given module.
– No. of components that can call A
– (shows how many places are using this module)

Eg – If modules A, B, C call function X → Fan-in = 3.

Interpretation :-
- High Fan-in → module is heavily used → good for reusability
- But changes to this module affect many others → risky.

(ii) Fan-out (FO) – No. of components that are called by A.
– No. of modules that given module calls.

If module X calls functions F1, F2 → Fan-out = 2.

Interpretation –
- High Fan-out → module depends on many modules → complex.
- More effort needed to understand and test.

(iii) Information flow complexity (IFC or "structural complexity")

$IF = (\text{Fan-in} \times \text{Fan-out})^2$

$IF = (FI \times FO)$.

Meaning –
- Measures overall complexity based on data movement.
- If FI or FO increases → complexity increases dramatically.
- Helps in estimating testing and maintenance

Example –

Suppose module M has

Fan-in = 3

Fan-out = 2

Then $IF = (FI \times FO)^2$

$= (3 \times 2)^2 = 36$

This module is moderately complex.

- Information flow metrics measures how information enters, flows through, and exists a module using Fan-in and Fan-out.

SOFTWARE RELIABILITY

→ Software reliability means how well software works without failing for a given time.

→ How long can the software run without crashing or giving wrong output?

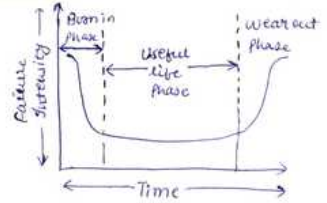
Importance –

(i) Ensures correct and consistent output
(ii) Reduce failures, bugs and downtime.
(iii) Improves user trust and satisfaction.
(iv) Essential for critical systems (Banking, Aviation, Healthcare).
(v) Reduce maintenance cost.

Wear-out phase – the stage where the device becomes old and stops working due to that wear and tear.

(1) HARDWARE RELIABILITY –

a) Hardware fails due to physical reasons (wear and tear, heat, dust)
↓
Things slowly get damage because of regular use.



Hardware Product

Burn-in phase
The early period of a hardware's life where weak or defective components fail quickly.

Useful life phase
The long middle period where hardware works normally and reliably with very few failures.

(b) Failure rate increases with time.

(c) Reliability improves by better materials.

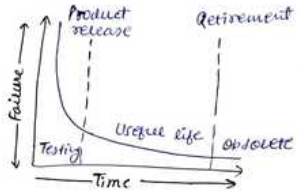
Eg- A Harddisk stop working after years → hardware failure.

Software Reliability -

(a) Software doesn't wear out physically.

(b) Failures happens due to bugs, wrong logic, or input errors.

(c) Failure rates depends on quality of code, not age.



- Obsolete - No longer in use, or outdated because something newer has replaced it.

Difference b/w Both -

Hardware Reliability	Software Reliability
(i) Failure are caused due to defects in design, production, and maintenance.	(i) Failure are caused due to defects in design.
(ii) Ability of hardware to work without physical failure	(ii) Ability of software to run without errors or crashes.
(iii) Reliability decreases over time as hardware wears out.	(iii) Reliability does not decreases with time because software doesn't wear out.
(iv) The Bathtub curve is used for failure rates apply.	(iv) There is no Bathtub curve for failure rates.
(v) Improved by using better hardware components and materials.	(v) Improved by better design, coding, testing and debugging.

FAULTS : (Bug / Error)

→ A fault is a mistake in the software's code or design.

→ It is something wrong inside the program, but it may not be visible immediately.

Eg- Writing a wrong formula in code as a fault

$a = b+c$ instead of $a = b-c$

FAILURE :

→ When the software does not work as expected because a fault is executed.

→ What the user sees as a problem.

Eg- Because of the wrong formula, the output is wrong → failure

Relationship b/w Faults and Failure

- A fault is inside the program.
- When executed, it causes a failure.

SOFTWARE RELIABILITY MODELS

→ Software reliability models are simple mathematical methods that help us predict :

how many bugs will appear,
how fast failures will reduce,
and how reliable the software will become after testing.

→ Describe how software failures occur over time and predict reliability.

τ Execution Time → Actual time the computer takes to run a program or perform a task

t Calendar Time → Total real-world time from start to end of a task, including waiting, delays, input/output, and other pauses.

→ The reliability of a program increases through faults correction and hence the failure intensity decreases.

BASIC MODEL – (Exponential Model)

→ This models assumes that failures happen randomly and the failure rate stays constant during testing.

→ As bugs are fixed, the software becomes reliable at a steady and slow rate

→ Useful for simple systems where failures happen unpredictably.

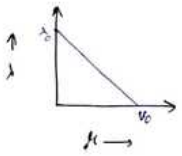
$$\lambda(\mu) = \lambda_0 \left[1 - \frac{\mu}{V_0}\right]$$

λ = failure intensity.

λ_0 = Initial failure intensity at the start of execution

V_0 = No. of failures experienced, if program is run for an infinite time period.

μ = Average or expected no. of failures experienced at a given point in time.



$$\frac{d\lambda}{d\mu} = \frac{-\lambda_0}{V_0}$$

[-ve sign signifies a decrementing trend in failure intensity]

LOGARITHMIC POISSION MODEL (Musa Model)

→ Failure rate decreases quickly at first, then slows down. (Because early bugs are found faster, later bugs take more time).

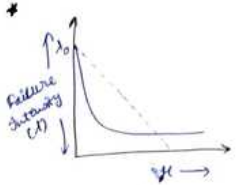
→ Mathematically -

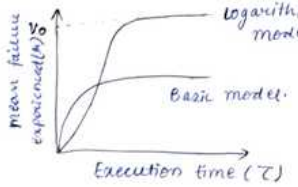
↳ failure rate decreases logarithmically.

→ Systems becomes more reliable but improvements slows down

$$\lambda(\mu) = \lambda_0 \exp(-\theta\mu)$$

θ = failure intensity decay parameter.





Mean failure experienced

$$\frac{d\lambda}{d\mu} = -\lambda_0 \theta \exp(-\mu\theta)$$

$$= -\theta\lambda$$

• At larger values of execution time, the logarithmic model will have larger values of failure intensity.

SOFTWARE QUALITY MODELS

→ Software quality models help us measure and understand software quality.

→ They tells us what makes software "good".

→ They define different quality characteristics like functionality, reliability, usability, efficiency, maintainability, and portability.

→ Common models include McCall's model, Boehm's model, and ISO 9126, which help developers understand whether the software meets user needs and is easy to maintain.

(i) McCall's Quality Models: Divides software quality into 3 groups: 1b

- Product Operation (how well it works)
- Product Revision (how easy to change)
- Product Transition (how easy to move to another system).

→ Focuses on things like correctness, reliability, usability, maintainability, portability.

→ Helps check if software works well for users and is easy for developers to maintain.

(ii) Boehm's Quality Models:

→ Similar to McCall's but more detailed.

→ Focuses on 3 main qualities:

- As-is Utility → How useful software is
- Maintainability → How easily it can be updated.
- Portability → How easily it can run in other environments.

→ Helps compare software quality from the user view + developer view.

(iii) ISO 9126 Quality Model:

→ A standard quality model with 6 key characteristics -

a) Functionality (does it do what it should?)
b) Reliability (does it work consistently?)
c) Usability (Is it easy to use?)
d) Efficiency (is it fast and resource-friendly?)
e) Maintainability (easy to fix / improve?)
f) Portability (easy to move to other systems?)

→ Very commonly used in industry to measure software product quality clearly.

CMM - Capability Maturity Model

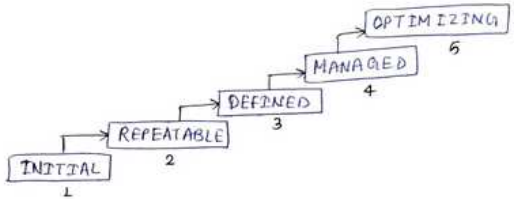
→ CMM is a model that tells how mature, organized, and controlled a ~~country~~ company's software development process is.

→ It shows how good a company is at building software - from beginners (level 1) to experts (Level 5).

→ The idea is simple:

- Better process = Better software quality.

CMM has 5 maturity levels:-



(i) Level 1 - Initial (Chaos stage)

- Work is Unorganized and unpredictable.
- No fixed process
- Success depends on individual people, not the system.
- Company works without any proper plan or rules.

(ii) Level 2 - Repeatable (Basic Management Stage)

- Company starts using simple rules that work again and again.
- Some basic processes are established.
- Past successful projects can be Repeated using the same methods.

(iii) Level 3 - Defined (Standard Process stage)

- All processes are documented, defined and standardized.
- Company follows proper written procedures.
- Everyone in the organization follows the same set of procedures.

(iv) Level 4 - Managed (Measured Stage)

- Company uses numbers and data to check if work is going well.
- Uses metrics (no. and data) to measure quality and performance
- Everything is monitored and controlled.

(v) Level 5 - Optimizing (Continuous Improvement stage)

- Company keeps improving and becomes the best version of itself.
- Focuses on continuous improvement
- The company uses new ~~jobs~~ tools, techniques, innovations, and feedback to improve processes.
- The goal is to prevent problems, not just fix them.

ISO 9001 - International standard

→ ISO 9001 is an International standard that tells organizations how to create and maintain a good quality management system (QMS).

→ It does not check the quality of the software itself - it checks whether the process used to build the software is good, consistent, and well managed.

- ISO 9001 ensures the company works in a systematic, organized, and quality-focused way.
- It helps companies deliver products that meets customer needs every time.

How ISO 9001 helps software companies

(i) Creating a structured development process.
(ii) Improving documentation.
(iii) Increasing consistency in work.
(iv) Preventing mistakes.
(v) Ensuring client requirements are correctly understood.

- Tracking changes and maintaining versions.
- Helping deliver high quality products on time.

★ Key requirements of ISO 9001.

An organization must :-

- Define processes clearly (who will do what).
- Document rules and procedures properly
- Train employees for their roles.
- Monitor and measure work progress.
- Handle customer complaints and fix issues.
- Do internal audits regularly.
- Review performance and plan improvements.

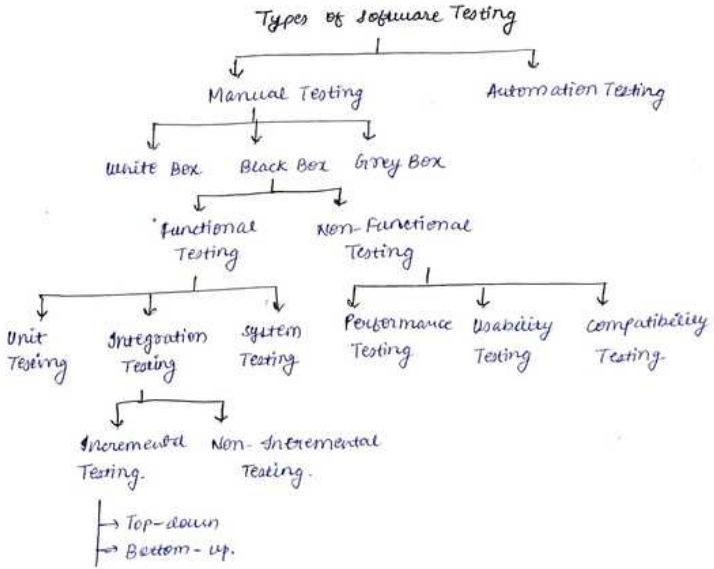
★ Benefits of ISO 9001 -

— Better quality and fewer defects.
— Higher customer satisfaction.
— Better communication.
— Smooth workflow and less confusion.

—— X ——

SOFTWARE TESTING -

- Software Testing is the process of verifying and validating whether the Software Product or Application is working as expected or not.
- It helps find bugs, mistakes, and missing requirements before the software is delivered.
- Testing compares expected results with actual results.
- It ensures software is reliable, safe and user-friendly.
- The main goal is to make sure the software meets customer needs and work without errors.



Software Testing Process -

→ It is a series of steps followed to check the software for errors and ensure it works correctly before delivery.

It includes the following stages -

(i) Requirement Analysis
- Test team studies the requirements of the software
- They understand what needs to be tested.
- Identify what is testable and what test data is needed.
  - Req: padho, samjho, kya test karna hai decide karo.

(ii) Test Planning
- Decide testing strategy (manual/ automation).
- Estimate time, cost and resources.
- Define test scope, tools, risks, and schedule
- Prepare the test plan document.

(iii) Test Case Design/ Development.
- Write test cases and test scenarios.
- Prepare test data.
- Review and update test cases.

(iv) Test Environment Setup
- Set up hardware, software, network, and tools.
- Ensure the environment ~~setup~~ matches the real user setup.
- Perform smoke testing

(v) Test Execution
- Execute test cases as per plan
- Compare actual result vs expected results.
- Mark test cases as Pass/Fail

(vi) Defect Reporting & Tracking
- Report bugs with detail.
- Track defect status (New, open, fixed, Re-tested, closed).
- Retest after fixes and perform regression Testing.

(vii) Test closure

- Check if exit criteria are met
- Prepare Test Summary Report (showing what was tested and results).
- Note down lesson learned for future projects.
- Store all test documents for future reference.

Functional Testing

→ Checks whether the software works correctly according to requirements.

→ It checks functions and features, not internal code.

→ Purpose - To ensure software gives correct output for given input.

What is Tested?

- Input fields
- Buttons and links
- Login and logout
- Calculations and Validations

What is not Tested?

- Speed of Software
- Performance
- Security
- Quality.

*Types of functional Testing -

- Unit Testing
- Integration Testing
- System Testing.

* Techniques

- Boundary Value Analysis
- Equivalence class Testing
- Decision Table Testing
- Cause Effect Graphing.

→ Functional Testing is also known as Blackbox Testing.

BOUNDARY VALUE ANALYSIS

(Black-box)

→ It is a functional^ testing techniques where test cases are designed by checking values at the edges (boundaries) of input ranges, because errors often occur at these points.

→ It means testing the minimum, maximum and values just below and above them instead of testing all possible values.

Why use BVA?

- Most defects happen at Boundary limits
- Reduces no. of test cases.
- Improves test coverage with fewer tests

Eg- Range = 40 to 80

Invalid Test case = Min value - 1 = 40 - 1 = 39

Invalid Test case = Max value + 1 = 80 + 1 = 81

Valid Test case = (Min + 1, Max - 1) = 41, 40, 79, 80
(Min, Max)

Q: 2024

(1) (k) (i) Age - form requires students to enter their ages which must be between 18 and 30 years inclusive.

(ii) No. of complete subjects → minimum of 5 and a max 10.

→ (i) Test case

Range = (18, 30)

No.	Input (Age)	Result
1.	18	✓ Accepted
2.	19	✓
3.	17	x Rejected
4.	30	✓
5.	29	✓
6.	31	x

(ii) Test case

No.	Input (subject)	Result
1.	5	✓
2.	4	x
3.	6	✓
4.	12	✓
5.	13	x
6.	11	✓

Q. Consider a program for determining the previous data ①

Input: Day, month, year with valid ranges as:

$1 \le$ month ≤ 12
$1 \le$ Day ≤ 31
$1900 \le$ Year ≤ 2000

} Possible o/p: Previous date or Invalid date

Design Boundary value Test case.

Test case No.	Month	Day (Nominal/valid value)	Year	Output	
1.	6	15	1900	14 June 1900	Year
2.	6	15	1901	14 June 1901	
3.	6	15	1906	14 June 1906	
4.	6	15	1999	14 June 1999	
5.	6	15	2000	14 June 2000	
6.	6	1	1960	31 May 1960	Day.
7.	6	2	1960	1 June 1960	
8.	6	30	1960	29 June 1960	
9.	6	31	1960	Invalid. date I/P	
10.	1	15	1960	14 Jan 1960	Month
11.	2	15	1960	14 Feb 1960	
12.	11	15	1960	14 Nov 1960	
13.	12	15	1960	14 Dec 1960	

I/P = x
Test cases = 4x+1
I/P = 3
Test cases = 4.3+1 = 13

Equivalence class Testing

- It is a functional (black-box) testing technique where input data is divided into groups (classes).
- Instead of testing every value, one value from each class is tested.
- Inputs are divided into valid and invalid equivalence classes.
- Any value in one class gives the same result, so testing one value is enough.
  - → Valid Equivalence class → Inputs that are accepted.
  - → Invalid Equivalence class → Inputs that are not accepted.

Advantage	Limitation
→ Fewer test case needed.	→ Boundary errors may be missed.
→ good for larger i/p ranges	→ Not suitable when ranges are uneven.

Eg ① Age field (18-30)

Valid class - Age between 18-30. (including 18 & 30)

Invalid class - Age < 18, Age > 30.

Boundary values check januri nahi hota

Test case No.	Input (Age)	Result / output
1.	25	✓
2.	16	X
3.	31	X

Eg ② Previous Date program

Range - $1 \le m \le 12$, $1 \le d \le 31$, $1900 \le$ year ≤ 2025.

Inputs → Day, month, year.

Output O1 → All invalid I/P → Previous date.
O2 → Invalid Date.

I/P is valid output is not valid

$I_1 \rightarrow 1 \le m \le 12$
$I_2 \rightarrow m < 1$
$I_3 \rightarrow m > 12$
$I_4 \rightarrow 1 \le d \le 31$
$I_5 \rightarrow d < 1$
$I_6 \rightarrow d > 31$
$I_7 \rightarrow 1900 \le y \le 2025$
$I_8 \rightarrow y < 1900$
$I_9 \rightarrow y > 2025$

Month	Date	year
6	15	1962
2	31	1962
6	15	1962
0	15	1962
13	15	1962
6	15	1962
6	-1	1962
6	32	1962
6	15	1962
6	15	1890
6	15	2026.

Decision Table Testing

→ Decision Table Testing is a functional (black-box) testing technique used when the output depends on multiple conditions and their combinations.

→ It represents conditions and actions in a table form.

→ Each column of the table represents one test case.

COMPONENTS

① Conditions – input rules (Yes/No, True/False)
② Actions – output or result
③ Rules – combinations of conditions.

Advantages	Limitation
① Covers all possible combinations.	① Table becomes large for many combinations.
② Reduces missing test cases	② Time consuming for complex system.

DECISION-TABLE

C_1: x,y,z are sides of Δ	N	Y							
C_2: x=y	—	Y				N			
C_3: x=z	—	Y		N		Y		N	
C_4: y=z	—	Y	N	Y	N	Y	N	Y	N
a_1: Not a Δ	✓								
a_2: Scalene									✓
a_3: Isosceles					✓		✓	✓	
a_4: Equilateral		✓							
a_5: Impossible			✓	✓		✓			

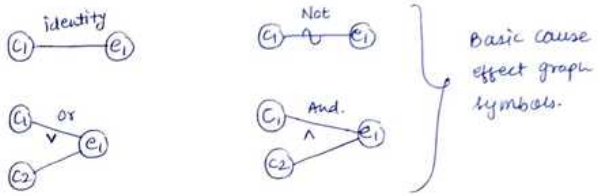
★ If all entries are binary (Y/N, T/F) then the decision table is called Limited Entry Decision Table.

★ If conditions can have several values then the decision table is called Extended Entry decision Table.

CAUSE EFFECT GRAPHING

→ It overcomes the limitation of Boundary value Analysis and Equivalence partioning because those techniques do not consider combinations of input conditions;
They basically consider only simple input conditions.

★ It is a method of generating test cases representing combinations of conditions.



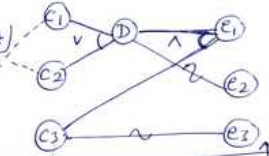
CAUSE → input condition
Effect → output condition

Advantages	Disadvantages
① Reduces the no. of test cases.	① Difficult to create for very complex systems.
② Helps detects logical defects.	② Not suitable for simple applications.

Q. The characters of column 1 must be an A or B. → If true, file update is made.
column 2 must be a digit

Causes: c_1: character in column 1 is A.
c_2: " " " 2 is B
c_3: " " column 2 is digit

Effect: e_1: update made.
e_2: message x
e_3: message y.



E = Exclusive
I = Inclusive
O = one and only one → one & only one
R = For c_1 to be one, c_2 must be one

• If a & b are digit → e_1
• c_1 & c_2 are not a & b → e_2 (Not a)
• c_3 is digit then and with e_1
digit not then e_3

BLACK BOX TESTING	WHITE-BOX TESTING
① The internal structure or Program or code is hidden and nothing is known about it	① The tester has knowledge about the internal structure or code or program of the s/w
~~② Focuses on how the program does~~	~~② Focuses on what the program does~~
② Focuses on what the software does	② Focuses on how the software does.
③ Mostly done by testers.	③ Mostly done by developers.
④ Done at system or Acceptance level	④ Done at unit level.
⑤ Implementation of code is needed.	⑤ Code implementation ~~of~~ is necessary.
⑥ Also called closed Testing	⑥ Also called clear Testing.
⑦ Eg- BVA, Equivalence partioning	⑦ Eg- Path Testing, Loop testing.

Structural Testing

- Software Testing that uses the internal design of the software for testing.
- Software Testing which is performed by the team which knows the development phase of the software, known as Structural testing.

Path Testing

- ~~It~~ is a structural (white-box) testing technique in which all possible execution path of a program are identified and tested to ensure complete logic coverage.
- Test cases are designed to execute all independent paths of a program at least one.

Limitations — Loops can create many paths.
Difficult for large programs.

→ Every possible execution path is tested
→ Helps find local and control-flow errors.
→ Uses control Flow Graph (CFG).

Steps

① Create control flow Graph ~~(CFF)~~ (CFG)
- Nodes → statements / blocks.
- Edges → flow of control

② Calculate Cyclomatic complexity.
—Tells number of independent paths.
- Formula :-
  V(G) = No. of decision points +1.

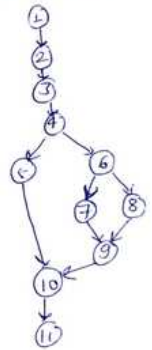
③ Identify Independent path.
↳ Each path must introduce a new edge.

④ Design one test case for each independent path.

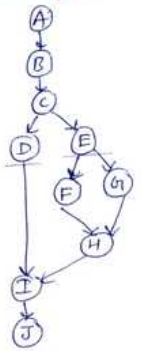
Eg- Program given :-

① Program Greatest
② Input (A)
③ Input (B)
④ If (A>B)
⑤ Then Print (A)
⑥ Else If (B>A)
⑦ Then Print (B)
⑤ Else
⑧ Print Both(=)
⑨ END if
⑩ End if
⑪ Exit
⑫ Exit

Flow Graph



Decision-to-decision Path graph (DD Path graph)



Cyclomatic Complexity → 2+1 = 3.

∴ 3 Independent path should be exist in DD Path graph

① ABCDEIJ ② ABCEFHIJ ③ ABCEGHIJ.

DATA FLOW TESTING

→ It is one of the white-box testing techniques.

→ It focuses on two points:-

* In which statement the variables are defined.
* In which statement the variables are used.

Main Idea:-

(a) Definition (def) → Variable gets a value

(b) Use (Variable is used).

(c) Kill → Variable value is destroyed or redefined.

Eg-

```
1) read a,b,c;
2) if (a>b)
3)     x = a+1
4)     print x;
5) else
6)     x=b-1
7)     print z;
```

Variables	Define	Use
a	1	2,3
b	1	2,5
c	1	NA
x	3,5	4
z	NA	6

Advantages -

① Finds Uninitialized variables.

② Detects unused or wrongly used variables.

Disadvantages

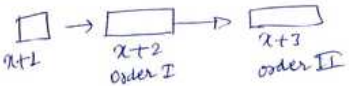
① Difficult to apply to large problems.

② Time-consuming.

Mutation Testing

→ It is a testing technique in which small changes (Mutations) are intentionally made in the program code to check the quality of test cases.

→ Multiple copies of a program are made, and each copy is altered, this altered copy is called a mutant.



• Mutants are then executed with test data. To determine whether the test data are capable of detecting the change between the original program and the mutated program

* original → I order → II order → III order →

• For specific test suit, mutants ~~are~~ can be either killed, live or equivalent.

UNIT TESTING

• Unit Testing is a level of software Testing in which individual units or components of a program (such as a function, method or class) are tested separately to check whether they work correctly.

* Tests the smallest part of the software
* Focuses on internal code logic.
* Usually done by developers.
* Performed before integration testing.

Advantages

① Finds bugs early.

② Easy to identify errors and fix errors.

③ Improves code quality.

④ Save times later in testing stages.

Limitations

① Cannot find integration issues.

② Requires coding knowledge.

③ Time consuming if units are many.

INTEGRATION TESTING

- Individual modules that have already been unit tested are combined and tested together to check whether they work correctly as a group.

→ Tests interaction b/w modules
→ Done after Unit testing.
→ Can be done by developers or testers.

Eg- Login module
Database module
Together to check whether login data is correctly stored in the database

Types of Integration testing.

① Top-down Integration - Testing starts from top modules and moves downward.
② Bottom-up Integration - Testing starts from lower-level modules and moves upward.
③ Big-Bang Integration - All modules are integrated and tested at once
④ Sandwich (Hybrid) Integration → combination of top-down and bottom-up testing.

Advantages
① Ensures smooth data flow between modules.
② Detect issues early in module interaction

Limitations
① Difficult to locate the exact faulty module
② Time-consuming for larger systems.

SYSTEM TESTING

- The entire software system is tested as one complete unit to check whether it works according to the given requirements.
- System Testing checks –
  - ↳ All features of the software
  - ↳ How different parts of the system works together
  - ↳ Whether the system behaves correctly in real-life situations.
- At this stage, software is tested just like a real user would use it.

* After Unit testing.
* After Integration testing.
When the full system is completely integrated.

System Testing performs by mostly Testers.
Sometimes QA Team
(Black-box Testing).

Eg – for an E-commerce website, system testing includes –
① User registration
② Login and Logout
③ Add to cart
④ Payment process etc
} all features are tested together

Types–
① Functional Testing
② Performance Testing
③ Security Testing
④ Usability Testing.

Advantages
* Find errors in the complete system.
* Ensures software meets user requirements.

Limitations
* Time consuming
* Errors are harder to locate

DEBUGGING

1. Debugging is the process of finding, identifying, and fixing errors (bugs) in a program after testing.
2. Testing only shows that an error exists, but debugging helps to locate the cause of the error and correct it.

Steps in Debugging -

(1) Identify the error
(2) Find the cause of the error
(3) Correct the error
(4) Test the program again

Importance of Debugging -

- Improves correctness of the program.
- Helps the software works as expected.

Testing	Debugging
(1) Testing is used to find errors in software.	(1) Debugging is used to fix errors in software.
(2) It shows that a problem exists.	(2) It finds where the problem is.
(3) Done before debugging.	(3) Done after testing.
(4) Can be done by testers.	(4) Mostly done by developers.
(5) Doesn't change the code	(5) Involves changing the code
(6) Focuses on checking behavior.	(6) Focuses on correcting logic.

Testing Tools

- Tools that are used for testing of software.
- Testing tools are software programs that help testers test software automatically and efficiently.

Why are they used?

(1) Saves time and effort
(2) Reduce manual work
(3) Useful for repeated testing.

Types of Testing tools -

(1) Static Testing -
- checks the software without running the program.
- Check code, documents, and requirements.
- find errors early, before execution.

(2) Dynamic Testing -
- checks the software by running the program
- Execute the software
  checks actual output vs expected output.

Eg - Automation tool - Selenium
Unit testing tool - JUnit
Performance tool - JMeter
Bug tracking tool - Jira

Testing standard

→ These are guidelines, rules, and best practices followed during software Testing to ensure the software is high quality, reliable, and well-documented.

(1) IEEE (Institute of Electrical and Electronics Engineers)
↳ Provides standard for Software Test documentation (IEEE 829)

(2) ISO 9001 (Quality Management System)
↳ International Organization for Standardization
↳ Defines standards for quality management.

(3) ISTQB - International Software Testing Qualifications Board)
↳ Provides ~~safety~~ testing guidelines and certification

SOFTWARE MAINTENANCE

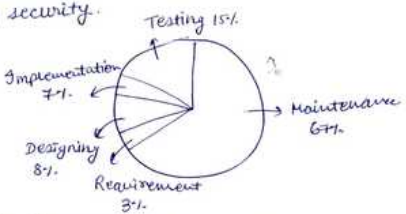
→ It refers to the process of modifying and updating a software system after it has been delivered to the customer.

Why –

① To fix bugs found after delivery.
② To add new features
③ To adapting to new hardware or software environments.
④ To improve performance and security.

Types :–

① Perfective Maintenance
→ Improving performance or adding new features.
② Preventive Maintenance
→ Preventing future problems by improving code structure
③ Adaptive Maintenance
→ Making changes to run software in a new environment.
④ corrective Maintenance
→ Fixing errors or bugs in software



Management of Maintenance

- Planning, organizing, and controlling software maintenance activities after the software is delivered, so that it continues to work properly.

Activities in this :–

① Receive maintenance requests.
② Analyze the problem or change request.
③ Estimate cost, time and resources
④ Plan and schedule maintenance work

⑤ Implement the changes.
⑥ Test the modified system.
⑦ Update modification.

Importance – Ensures software remains reliable and useful.

Maintenance Process

→ It is the step-by-step method followed to modify, fix or improve software after it is delivered to users.

STEPS –

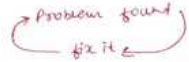
① Problem Identification – User reports an error or requests a change.
② Problem Analysis – Understand the problem and find its cause
③ Change Request – A formal request is made for modification
④ Impact Analysis – Analyze effect on cost, time, and system
⑤ Implementation – Make required changes in the software.
⑥ Testing – Test the system after changes.
⑦ Documentation Update – Updates manuals and records.
⑧ Release – Deliver the updated software.

Maintenance Models.

→ Describe how software maintenance activities are organized and carried out after the software is delivered.

4 Models

① Quick-fix Models



– Used when a small change or bug fix is needed quickly.
– Changes are made directly to the code.
– disadvantage – code quality may degrade over time.

② Iterative Enhancement Model

–



- No availability of full documentation.
- Maintenance team can't analyze the existing product completely

(3) Reuse Oriented Model.

- Uses existing components or modules for maintenance.
- Saves time and cost.
- Works well for large systems.

(4) Boehm's Maintenance Model.

- Proposed by Barry Boehm
- Focuses on cost and effort estimation in maintenance.
- Explain how much effort goes into:
  - Understanding the program.
  - Making changes
  - Testing and documentation.

(5) Taute's Maintenance model.

- Proposed by Taute.
- Focuses on type of maintenance activities.
- Divides maintenance work into -
  - Corrective
  - Adaptive
  - Perfective

Regression Testing

→ It is a type of software Testing that verifies that changes made to the system, such as bug fixes or new features, do not impact previously working functionality.

→ It ensures old functionality is not broken by new changes

Why is it important?

→ Prevents new bug in existing features.

→ Maintain software stability.

Role of Regression in Agile

→ In Agile model, software is developed in small iterations (sprints) with frequent changes.

Regression testing plays a very important role to ensure that new changes do not break existing features.

Reverse Engineering	Re-engineering
(1) Understands an existing system	(1) Improves or modifies an existing system
(2) Focuses on analyzing the code	(2) Focuses on changing the system
(3) No change is made to the software	(3) Software is modified or rebuilt
(4) Used to get design or documentation.	(4) Used to improve performance or maintainability
(5) Output is documentation or design	(5) Output is an improved software system
(6) First step before re-engineering	(6) Done after reverse engineering.

Configuration Management

→ It is the process of controlling and managing changes made to software during development and maintenance.

→ It manages - Source code, Documents, versions, changes and updates.

Why Important?

→ Avoid confusion between versions.

→ Keeps track of who changed what and when

→ Helps in teamwork and maintenance

Documentation

→ Written information that explains the software, its use, design, and maintenance.

Types –

① User documentation - how to use the sft
② System " = design, architecture, code details.
③ Maintenance " helps future updates and fixes.

Verification	Validation
① Are you building it Right?	① Have you build the right thing?
~~② Check whether an artifici~~	
② Focuses on requirements, design, and implementation correctness.	② Focuses on functional behavior.
③ Verification is for prevention of errors.	③ Validation is for detection of errors.
④ Done during development	④ Done after development.
⑤ Does not involve running the program.	⑤ Involves running the program.
⑥ Uses reviews, inspections, walkthroughs.	⑥ Uses testing like system or acceptance testing.
⑦ Ensures software meets specified requirements	⑦ Ensures. software meets user needs.